

Para empezar utilice el prompt que el profesor nos proporciono en clase el cual es el siguiente:

Actúa como asistente de programación para una práctica de Estructura de Datos en Python. Contexto:

Ya tengo un proyecto desarrollado en clase con Python, programación orientada a objetos y una interfaz gráfica creada con Qt Designer. En el proyecto ya existe una clase Stack con los métodos básicos de una pila. También existe una ventana o formulario donde se capturan datos y se muestran resultados, así como la clase que realiza la carga y la detección del evento para convertir la expresión.

Objetivo:

Necesito agregar al proyecto existente la funcionalidad para convertir una expresión infija a expresión posfija utilizando la estructura de pila ya creada en clase. Necesito se genere una clase que contenga una pila y un método para llevar acabo la conversion.

Despues analize la clase que me proporciono la IA la cual le pedi que la llamara Aplicaciones y me proporciono la siguiente:

```
class Aplicaciones:
    def __init__(self):
        # Creamos la instancia de tu pila dentro de la clase Aplicaciones
        self.stack = Stack()

        # Diccionario de precedencia de operadores
        self.precedence = {
            '+': 1,
            '-': 1,
            '*': 2,
            '/': 2,
            '^': 3
        }

    def _get_precedence(self, operator):
        """Método auxiliar interno para medir la prioridad."""
        return self.precedence.get(operator, 0)

    def convert_infix_to_postfix(self, infix_expression: str) -> str:
        """
        Método que realiza la conversión utilizando tu estructura de Stack.
        """
        postfix_result = []
        tokens = infix_expression.replace(" ", "") # Limpiamos espacios

        for token in tokens:
            # 1. Si es letra o número (operando)
            if token.isalnum():

```

```

        postfix_result.append(token)

        # 2. Si es paréntesis de apertura
        elif token == '(':
            self.stack.push(token)

        # 3. Si es paréntesis de cierre
        elif token == ')':
            while not self.stack.is_empty() and
self.stack.top_of_stack() != '(':
                postfix_result.append(self.stack.pop())

            if not self.stack.is_empty() and self.stack.top_of_stack()
== '(':
                self.stack.pop() # Eliminamos el '(' de la pila

        # 4. Si es un operador (+, -, *, /, ^)
        else:
            while (not self.stack.is_empty() and
                    self._get_precedence(self.stack.top_of_stack()) >=
self._get_precedence(token)):
                postfix_result.append(self.stack.pop())

            self.stack.push(token)

        # 5. Vaciar los operadores restantes
        while not self.stack.is_empty():
            postfix_result.append(self.stack.pop())

        return "".join(postfix_result)

```

La lleve a revision con el profesor para ver si podia usarla despues de que me dijo que le parecia bien entonces despues de ver que se ajustaba a lo que necesitaba la copie y pegue en mi archivo de Aplicaciones.py el cual cree previamente.

Despues cree la interfaz grafica en Qt Designer la cual guarde en mi carpeta .ui despues cree un archivo en mi carpeta load para cargar mi interfaz el cual llame load_aplicaciones.py aquí cargue lo que me dio la IA ya solo puse el nombre de mi interfaz y cambie lo necesario de nombres que tenia en mis etiquetas de la interfaz para que coincidieran con las de IA

```

class LoadAplicaciones(QDialog):
    def __init__(self):
        super().__init__()
        uic.loadUi('ui/Nombre del archivo.ui', self)
        self.btn_convertir.clicked.connect(self.convert_infix_to_postfix)

```

```

def convert_infix_to_postfix(self):
    # Instanciamos tu nueva clase Aplicaciones
    procesador = Aplicaciones()

    # Ejecutamos el método de conversión
    texto_posfijo =
procesador.convert_infix_to_postfix(self.txt_infijo.text())

    # Mostramos el resultado en la interfaz gráfica
    self.lbl_resultado.setText(texto_posfijo)

```

Casi para Finalizar modifique mi archivo load_menu.py para que pudiera cargar mi interfaz añadiendole lo necesario

```

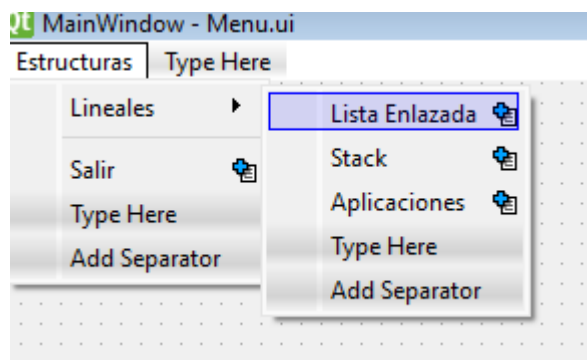
self.actionAplicaciones.triggered.connect(self.load_aplicaciones)
y def load_aplicaciones(self):

    aplicaciones = LoadAplicaciones()
    aplicaciones.exec_()

```

ademas de Importar la clase `from load.load_aplicaciones import LoadAplicaciones`

Cree las opcion en mi menu existente de aplicaciones



Al final hize las pruebas correspondientes .